

Making a robot use Google Flow without losing its mind.

A plain-language tour of our video generation pipeline – what broke, what we fixed, how parallel generation works, and where you come in.

THE ONE-SENTENCE VERSION

We drive Google Flow's website with an automated browser to render short-form video clips. Google's site fights back – so every layer of our system is built to retry, recover, or fail with a message a human can act on.

1 Our goals & the process we run	p. 2
2 The system in one picture · why this is hard	p. 3
3 The reliability war – every failure & its fix	p. 4
4 Parallel generation – checkout lanes	p. 5
5 What we tried & improved · recurring errors	p. 6
6 Errors that talk like humans · open risks	p. 7
7 Your part: ship, deploy, test	p. 8

1 • OUR GOALS

What we are actually trying to build

Carnivore turns written scripts into finished short-form videos – carnivore-diet storytelling for social – with as little human babysitting as possible. Concretely, four goals rank above everything:

1

RELIABLE

A queued video either **finishes** or **tells you exactly why not**. No silent stalls, no infinite retries, no burned credits on hopeless runs.

2

FAST

Clips render **in parallel**, so a 5-clip video takes roughly one clip's time – not five.

3

LEGIBLE

Every failure surfaces as a **plain-language popup** with the one action that fixes it. Anyone on the team can operate this without reading code.

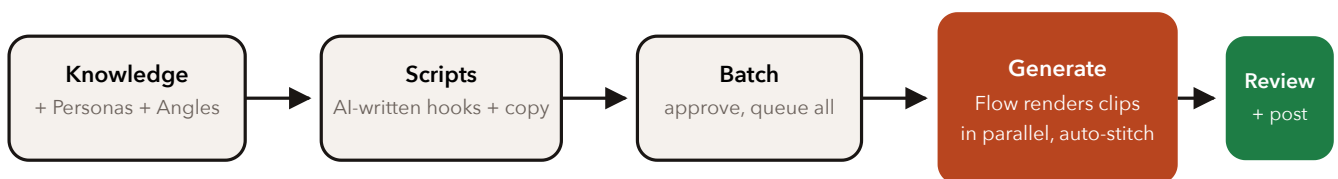
4

HANDS-OFF

Merge to **main** and the system tests, deploys, and health-checks itself. Production is never hand-edited.

1B • THE PROCESS WE RUN

From idea to posted video



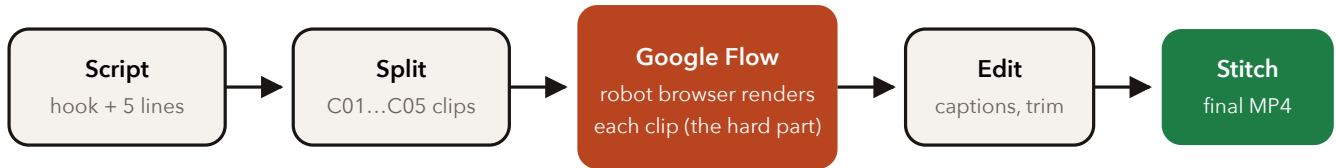
Steps 1-3 are cheap and instant (LLM text: DeepSeek / Grok / Venice). Step 4 is the expensive, fragile one – which is why the rest of this document is almost entirely about making **Generate** bulletproof.

DIVISION OF LABOR

Humans decide **what to say** (knowledge, personas, approving scripts). The system handles **everything after the approve button** – rendering, retrying, recovering, stitching – and reports back in plain language.

Script in, stitched video out

A script is split into 4-6 short clips. Each clip is rendered by Google Flow (Veo), downloaded, then the set is edited and stitched into one vertical video. Hold this picture – everything else hangs off it.



Runs on one server (62.83.10.231) as five Docker containers: the web app, the database, the **flow-worker** (the robot browser), the **reel-runner** (the queue processor), and a reverse proxy.

2B · WHY THIS IS HARD

Know your adversary

There is no official Flow API. Our worker opens a real Chrome, logs into a real Google account, clicks real buttons. That means our failures come in **two very different flavors** – and telling them apart is 80% of the reliability game:

FLAVOR A – OUR BUGS

Selectors drift, sessions get stale, retries loop.
Fixable in code. We fixed a stack of these – next page.

FLAVOR B – GOOGLE PUSHING BACK

Credit walls, session revocations, rate-limit "holds" after heavy use. **Not fixable in code.**
We can only detect, pace, and tell the user the truth.

REMEMBER THIS ONE THING

When generation fails, first ask: **is this us, or is this Google?** The error popup now answers that for you – that's what the error taxonomy is for (p. 5).

Every failure we hit – and its fix

This is the campaign log from hardening week. Each row was a real outage. Read it once now; it will save you hours of re-diagnosis later.

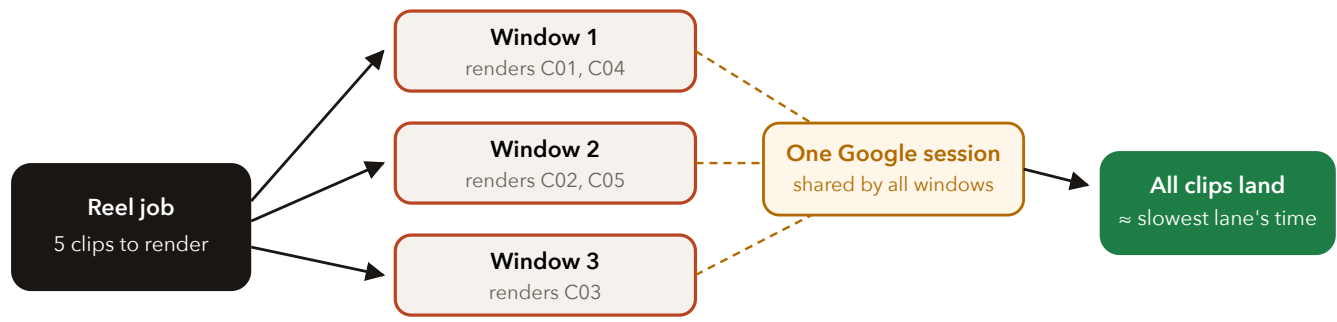
SYMPTOM YOU'D SEE	ROOT CAUSE	FIX (DEPLOYED)	NOW
"Flow has no character named African Elder" – every run dead on arrival	Missing character/photo was treated as a fatal error before generation even started	Character is now optional – worker skips attach and generates anyway	FIXED
Clip renders, downloads... then "speech did not match", retry ×10, credits burned	Speech verification needs a transcription API key. No key was configured – so verification could never pass	Transcription disabled – every rendered take is accepted as-is	FIXED
"Browser has been closed" seconds into every attempt	A zombie job restored from an old checkpoint fought new jobs for the same Chrome profile – they killed each other's browsers	Stale job state wiped; stale Chrome locks cleared	FIXED
Jobs sit "queued" forever, nothing happens at all	The queue processor container (reel-runner) was created but never started after a bad deploy	Started; CI deploy now health-checks every service	FIXED
Retried forever on a hopeless job	The recovery counter was never incremented – the "8 attempts max" gate never triggered	Counter wired up; hard stop at 8 with a clear final error	FIXED
8 rapid-fire failures in 3 minutes against a rate-limited account	Retries didn't respect Google's cool-down – hammering makes holds longer	Cool-down failures now wait 5 minutes between attempts	FIXED
"Flow session appears signed out"	Google expired/revoked the login (Flavor B)	Clear popup → re-sign in via Settings. Human action required	BY DESIGN
"Cost warning icon – generation will not start"	Google's soft hold after heavy usage on one account (Flavor B)	Paced retries ride it out; message says to wait or switch account	MANAGED

THE PATTERN BEHIND EVERY FIX

Never block on something optional. Never retry fast against a wall. Never fail silently. **Generate if at all possible; otherwise say exactly what a human should do.**

Think supermarket checkout lanes

One cashier serving five customers in a row is slow. Five lanes serve them at once. Clips are the customers; browser windows are the lanes.



Failure isolation: one clip failing does NOT abort its siblings – you get partial results, marked Incomplete. Checkpoints save every finished clip – a crash or restart resumes, it never re-pays for done work.

The two lane modes

SITUATION	WHAT THE WORKER DOES	RISK LEVEL
2+ Google accounts connected	One lane per account. Fully isolated sessions – Google sees normal usage per account.	PREFERRED
1 account (us, today)	Up to 3 windows share that account's session (was 8 – we lowered it after a rate-limit hold).	WATCHED

WHY THE UI LOOKS "SEQUENTIAL" SOMETIMES

All lanes write into one progress log. The "Creating C02..." label shows whichever clip logged **last** – so it hops around while, in reality, all clips render at once. Don't let it fool you: we verified 5 clips dispatching simultaneously across 5 windows.

The journey – each attempt taught us something

The honest iteration log – it explains *why* the system is shaped this way:

WE TRIED...	WHAT HAPPENED	SO WE IMPROVED TO...
Strict character matching (persona photo required)	Every batch died on "no character named X" before spending a single credit	Fallback persona → then character fully optional : generate with voice + prompt only
Speech verification (transcribe each take, compare to script)	No transcription key configured → every <i>good</i> take rejected, 10 retries, credits burned	Removed transcription – takes accepted as rendered; revisit when a key exists
Instant retries after failure	8 recovery attempts fired in ~3 minutes against Google's rate-limit wall – making the hold longer	Cool-down failures now wait 5 minutes between attempts
8 parallel windows on one account	Fastest config – and it tripped Google's "cost warning" usage hold the same day	Capped at 3 windows ; real fix is a second account
Restoring interrupted jobs on worker boot	A stale "zombie" job fought new jobs for one Chrome profile – mutual browser murder	Stale state wiped; deploys health-check every service
Manual deploys (rsync + ssh by hand)	One rsync flag deleted the server's secrets; a container was left never-started	CI/CD : push → tests → deploy → health verification

THE META-LESSON

Every "safety check" we removed blocked more good videos than bad ones. Keep a check only when it catches real failures cheaply.

Errors we keep encountering – learn these on sight

VERBATIM MESSAGE	WHAT IT REALLY MEANS	STATUS
All Flow accounts failed. Flow account: ...	Wrapper – the truth is the sentence <i>after</i> it.	RECURRING
Flow session appears signed out (.../about)	Google logged us out. Re-sign in via Settings.	RECURRING
...cost warning icon and will not start...	Usage hold; balance fine. Paced retries wait it out.	RECURRING
No Flow account is signed in with credits	No usable account (expired / resting / no credits).	RECURRING
Flow has no character named X...	Old hard-fail – character is optional now.	ELIMINATED
speech did not match (retry N/10)	Old verification loop – transcription removed.	ELIMINATED
...browser has been closed	Jobs fighting over one Chrome profile – if seen, check for zombie worker jobs.	ELIMINATED

The taxonomy: code → popup

Every failure the worker sees carries a machine code. One module (`flow-failure.ts`) translates codes into popups with a headline, one actionable sentence, and a button. No more cryptic stack traces for users.

CODE	POPUP HEADLINE	WHAT THE USER DOES
CREDITS	"Out of Flow credits"	Add credits or connect another account → Settings
EXPIRED	"Flow session expired"	Re-sign in → Settings
BLOCKED / hold	"Flow paused this account"	Wait ~20 min, or use another account
RECOVERY_EXHAUSTED	"Gave up after retries"	Fix the underlying cause, hit Regenerate (resets the budget)
<i>partial success</i>	Amber "Incomplete" badge	Regenerate only the missing clips – done ones are kept

THE RETRY BUDGET, IN ONE BREATH

Each clip: up to 10 render attempts inside the worker. Each job: up to 8 recovery cycles in the app – each cycle paced 5 minutes apart if Google is holding us. User's "Regenerate" resets everything.

Nothing loops forever anymore.

6B · OPEN RISKS

What can still bite us

1

GOOGLE ACCOUNT

All parallelism shares one session. Heavy days trip Google's usage holds – it happened Sept 15 and stalled a whole batch.

0

TRANSCRIPTION KEYS

Speech QA is off. If Veo mangles a line, nobody catches it before stitch. Acceptable for now – revisit when quality matters more than throughput.

3

WINDOWS PER ACCOUNT

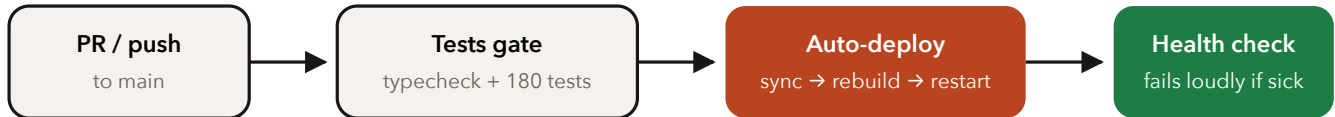
Our compromise between speed and not looking like a bot farm. Tune down if holds recur.

THE ONE BIG RECOMMENDATION

Connect a second Google Flow account. It removes the shared-session rate-limit exposure, doubles safe throughput, and a hold on one account no longer stalls a batch. Highest-leverage 30 minutes anyone can spend on this system.

Ship it, watch it deploy itself, test it

The repo is `github.com/johnpaulgarcia/carnivore`. CI/CD is live and proven – a one-line footer change went from `git push` to running in production in under 3 minutes.



- **Merging to `main` deploys to the server automatically.** No manual steps. The server's secrets (`.env`) are never touched by deploys.
- PRs run the same test gate without deploying – safe to open early and often.
- If the post-deploy health check fails, the Actions run goes red with container logs attached.

Try it right now

1. Open the app: `http://62.83.10.231`
2. Sign in with the admin access code below
3. Open **Videos** → pick a video → hit **Regenerate** and watch the live progress log
4. Make a tiny change on a branch, merge to `main`, and watch the Actions tab deploy it

ADMIN ACCESS CODE

`carni-admin-f25742`

Treat it like a password – don't commit it, don't paste it in public channels.

WELCOME ABOARD

You now know more about this system's failure modes than anyone who hasn't lived them. When something breaks, start at the popup message – it was built to tell you the truth.